BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



TIỂU LUẬN MÔN HỌC
PHÁT TRIỂN ỨNG DỤNG DI DỘNG

ỨN DỤNG EDGE AI TRONG HỆ THỐNG CHO THÚ CƯNG ĂN THÔNG MINH

Giáo viên hướng dẫn: Th.S. Huỳnh Hoàng Hà
Nhóm thực hiện: 06
Mã lớp học: MBAD436364

TP. Hồ Chí Minh, tháng 09 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



TIỂU LUẬN MÔN HỌC
PHÁT TRIỂN ỨNG DỤNG DI DỘNG

ỨN DỤNG EDGE AI TRONG HỆ THỐNG CHO THÚ CƯNG ĂN THÔNG MINH

GVHD: Th.S. Huỳnh Hoàng Hà

SVTH: Nhóm 06

1. Hoàng Ngọc Dung 23139006
2. Hoàng Mạnh Hùng 23139017
3. Trần Hữu Dương 23139009
4. Phạm Quang Phúc 23139034
5. Đoàn Minh Duy Bình 23139005

TP. Hồ Chí Minh, tháng năm 2026

DANH SÁCH NHÓM VIẾT BÁO CÁO CUỐI KỲ

HỌC KỲ 1 NĂM HỌC 2026–2027

1. **Mã lớp môn học:** MBAD436364
2. **Giảng viên hướng dẫn** : Th.S. Huỳnh Hoàng Hà
3. **Tên đề tài:** Ứng dụng edge AI trong hệ thống cho thú cưng ăn thông minh
4. **Danh sách nhóm và phân công nhiệm vụ**

STT	Họ và tên sinh viên	MSSV	Nội dung thực hiện	Ký tên
1	Hoàng Ngọc Dung	23139006	Phát triển mô hình Edge AI, huấn luyện, đánh giá, quantization và triển khai trên ESP32-S3 CAM.	
2	Hoàng Mạnh Hùng	23139017	Thiết kế hệ thống, xây dựng phần cứng và phát triển ứng dụng React Native kết nối Firebase.	
3	Phạm Quang Phúc	23139034	Phát triển ứng dụng, Firebase, chức năng quản lý cho ăn, lịch sử và kiểm thử hệ thống.	
4	Trần Hữu Dương	23139009	Chuẩn bị dataset, hỗ trợ huấn luyện mô hình, triển khai Edge AI và tích hợp điều khiển cho ăn.	
5	Đoàn Minh Duy Bình	23139005	Lắp ráp phần cứng ESP32-S3 CAM, servo, load cell/HX711; kiểm thử và tối ưu hệ thống.	

Nhận xét của giáng viên

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Ngày …. tháng …. năm 2026

Giảng viên chấm điểm

LỜI CAM ĐOAN

Chúng em xin cam đoan đề tài nghiên cứu là trung thực. Những kết quả số liệu, thông tin phục vụ cho quá trình xử lý và hoàn thành bài nghiên cứu đều được thu thập từ các nguồn khác nhau, có ghi rõ nguồn gốc. Tất cả các thông tin số liệu, bảng, hình ảnh và kết quả nghiên cứu được đều được trình bày một cách minh bạch và chân thực, không chứa bất kỳ sự biến đổi hay chỉnh sửa nào có thể làm ảnh hưởng đến tính chính xác của kết quả.

Các nguồn dữ liệu và thông tin được sử dụng trong nghiên cứu đã được thu thập từ các nguồn đáng tin cậy và đã được ghi rõ nguồn gốc. Bất kỳ tài liệu tham khảo, nghiên cứu trước đây hoặc ý kiến cá nhân được sử dụng đều đã được đặc tả một cách rõ ràng để đảm bảo tính minh bạch và khách quan. Chúng em xin chịu hoàn toàn trách nhiệm, kỷ luật của bộ môn và nhà trường nếu như có bất cứ vấn đề nào.

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến Th.S. Huỳnh Hoàng Hà giảng viên bộ môn Lập trình ứng dụng di động**.** Trong suốt quá trình học tập, Thầy đã rất tâm huyết dạy và hướng dẫn cho chúng tôi nhiều điều bổ ích trong môn học và kỹ năng làm một bài nghiên cứu để chúng tôi có đủ kiến thức thực hiện bài báo cáo này.

Tuy nhiên vì kiến thức bản thân còn nhiều hạn chế và sự tìm hiểu chưa sâu sắc nên không tránh khỏi những thiếu sót. Mong Thầy sẽ châm chước và cho chúng em những lời góp ý để bài nghiên cứu của chúng tôi sẽ hoàn thiện hơn. Một lần nữa, chúng tôi xin gửi lời cảm ơn sâu sắc đến Thầy và chúc Thầy luôn mạnh khỏe, hạnh phúc và thành công trong sự nghiệp.

TP. Hồ Chí Minh, tháng năm 2026

MỤC LỤC

DANH MỤC HÌNH ..... **ix**
DANH MỤC CÁC CHỮ VIẾT TẮT ..... **x**
CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI **1**
1. Tình hình nghiên cứu **1**
2. Đặt vấn đề **2**
3. Mục tiêu của đề tài **3**
4. Nội dung nghiên cứu **5**
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT **7**
2.1. Tổng quan hệ thống IoT và Edge AI **7**
2.2. ESP32-S3 và ESP32-S3 CAM **7**
2.3. Cơ sở lý thuyết về phân loại ảnh **7**
2.4. Mô hình AI nhẹ cho Edge Device **9**
2.5. Tối ưu mô hình trên ESP32-S3 ..... **10**
2.6. Servo Motor và cơ cấu cấp thức ăn **11**
2.7. Load Cell và HX711 ..... **11**
2.8. Firebase **11**
2.9. React Native **13**
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG **16**
3.1. Yêu cầu hệ thống **16**
3.1.1. Yêu cầu chức năng ..... **Error! Bookmark not defined.**
3.1.2. Yêu cầu phi chức năng ..... **Error! Bookmark not defined.**
3.2. Kiến trúc tổng thể hệ thống **16**
3.3. Thiết kế phần cứng **23**
3.3.1. ESP32-S3 CAM 23
3.3.2. Camera 23
3.3.3. Servo Motor 23
3.3.4. Load Cell và HX711 23
3.3.5. Nguồn cấp 23
3.3.6. Sơ đồ kết nối phần cứng 23
3.4. Thiết kế mô hình Edge AI **23**

3.5. Thiết kế chức năng cho ăn theo loài.....23
3.6. Thiết kế thuật toán cấp thức ăn 24
3.7. Thiết kế điều khiển dựa trên phản hồi khối lượng.....27
3.8. Thiết kế Firebase 30
3.9. Thiết kế ứng dụng React Native.....30
CHƯƠNG 4: PHÂN TÍCH KẾT QUẢ.....31
4.1. Môi trường thực nghiệm.....31
4.2. Kết quả mô hình AI.....31
4.3. Kết quả triển khai Edge AI.....31
4.4. Kết quả Firebase và ứng dụng 31
KẾT LUẬN 32
TÀI LIỆU THAM KHẢO.....32

DANH MỤC HÌNH

DANH MỤC CÁC CHỮ VIẾT TẮT

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI

1. Tình hình nghiên cứu

Trong đời sống hiện đại, chó và mèo không chỉ được nuôi để trông giữ nhà mà còn trở thành thành viên trong nhiều gia đình. Vì vậy, việc bảo đảm chế độ dinh dưỡng phù hợp có ảnh hưởng trực tiếp đến sức khỏe, thể trạng và chất lượng sống của thú cưng. Các hướng dẫn dinh dưỡng của AAHA và WSAVA đều nhấn mạnh rằng khẩu phần cần được đánh giá theo từng cá thể, dựa trên loài, khối lượng, giai đoạn sống, mức độ vận động và tình trạng sức khỏe; việc cho ăn thiếu kiểm soát có thể dẫn đến thừa cân, béo phì hoặc thiếu dinh dưỡng. Tuy nhiên, chủ nuôi không phải lúc nào cũng có mặt tại nhà để cho thú cưng ăn đúng giờ và đúng lượng. Lịch học tập, làm việc hoặc những chuyến đi ngắn ngày dễ làm gián đoạn thói quen cho ăn. Phương pháp cho ăn thủ công phụ thuộc hoàn toàn vào người chăm sóc, trong khi các máy cho ăn hẹn giờ thông thường chủ yếu chỉ mở cơ cấu cấp thức ăn theo thời gian cài đặt. Các thiết bị này thường chưa xác nhận được con vật nào đang tiếp cận, lượng thức ăn đã cấp thực tế là bao nhiêu hoặc lần cho ăn có diễn ra thành công hay không.

Trong gia đình nuôi nhiều thú cưng, hạn chế trên trở nên rõ rệt hơn. Một con vật có thể ăn nhầm khẩu phần của con khác; thú cưng lạ cũng có thể tiếp cận máng ăn; đồng thời sự cố kẹt cơ cấu, hết thức ăn hoặc mất kết nối có thể không được phát hiện kịp thời. Do đó, hệ thống chỉ điều khiển theo bộ hẹn giờ chưa đủ để đáp ứng yêu cầu chăm sóc chính xác và có khả năng kiểm chứng. Cần kết hợp cơ cấu cấp thức ăn với Load Cell, cảm biến hiện diện và camera để xác nhận đối tượng, đo lượng thức ăn và lưu lại kết quả sau mỗi lần hoạt động. Sự phát triển của Internet vạn vật tạo điều kiện liên kết bộ điều khiển, cảm biến, cơ sở dữ liệu đám mây và ứng dụng di động thành một hệ thống thống nhất. Nghiên cứu của Castillo-Arceo và cộng sự cho thấy việc kết hợp IoT, cảm biến khối lượng, cảm biến tiếp cận, camera và mô hình học sâu trong máy cho thú cưng ăn có thể hỗ trợ nhận diện đối tượng và kiểm soát khẩu phần [3]. Đối với đề tài này, ứng dụng di động giúp người dùng thiết lập lịch, ra lệnh cho ăn, theo dõi trạng thái và xem lịch sử; Firebase Realtime Database đóng vai trò đồng bộ dữ liệu giữa ứng dụng và thiết bị gần như theo thời gian thực. Bên cạnh kết nối IoT, trí tuệ nhân tạo giúp hệ thống chuyển từ cơ chế cho ăn tự động đơn thuần sang cơ chế có khả năng nhận biết đối tượng. Camera trên ESP32-S3 CAM thu nhận hình ảnh, mô hình AI xác định chó hoặc mèo và

trả về độ tin cậy trước khi hệ thống quyết định cấp thức ăn. Tuy nhiên, vi điều khiển có tài nguyên bộ nhớ và khả năng tính toán hạn chế, vì vậy mô hình phải được tối ưu về số tham số, dung lượng và thời gian suy luận. Dự án ESP-Detection của Espressif cung cấp các mô hình phát hiện đối tượng siêu nhẹ dựa trên YOLOv11 và quy trình triển khai cho ESP32-S3, cho thấy hướng xử lý AI tại biên là khả thi đối với phần cứng nhúng [5]. Các nghiên cứu về triển khai mô hình phát hiện đối tượng trên ESP32-CAM cũng khẳng định nhu cầu tối ưu mô hình khi xử lý ảnh theo thời gian thực.

Từ những vấn đề trên, việc xây dựng hệ thống cho thú cưng ăn thông minh kết hợp ứng dụng di động, cơ sở dữ liệu đám mây, cảm biến và AI là cần thiết. Đề tài hướng đến một giải pháp có chi phí hợp lý, dễ sử dụng, cho phép người nuôi kiểm soát việc cho ăn từ xa nhưng vẫn có cơ chế tự động xác nhận đối tượng và lượng thức ăn. Hệ thống đồng thời tạo ra dữ liệu lịch sử để người dùng theo dõi thói quen ăn uống, phát hiện tình trạng bất thường và cải thiện quá trình chăm sóc thú cưng.

2. Đặt vấn đề

Bài toán đặt ra là thiết kế một hệ thống có khả năng cho chó và mèo ăn đúng thời điểm, đúng khẩu phần và đúng đối tượng, đồng thời cung cấp thông tin giám sát cho người dùng trên điện thoại. Hệ thống phải phối hợp giữa ba khối chính: ứng dụng di động React Native, nền tảng Firebase và thiết bị nhúng gồm ESP32, ESP32-S3 CAM, servo, Load Cell, HX711 cùng cảm biến phát hiện. Mỗi khối thực hiện một nhiệm vụ riêng nhưng phải trao đổi dữ liệu thống nhất để lệnh cho ăn có thể được theo dõi từ lúc tạo đến khi hoàn tất.

Để giải quyết bài toán tổng thể, đề tài tập trung vào các yêu cầu sau:

- Quản lý tài khoản và hồ sơ thú cưng, bao gồm loài, khối lượng, loại thức ăn và khẩu phần khuyến nghị.
- Cho phép người dùng ra lệnh cho ăn thủ công hoặc thiết lập lịch cho ăn trên ứng dụng di động.
- Nhận diện chó hoặc mèo từ hình ảnh camera, kèm theo độ tin cậy để hỗ trợ quyết định cho ăn.
- Điều khiển servo cấp thức ăn, sử dụng Load Cell và HX711 để đo lượng thức ăn thực tế sau mỗi lần cấp.
- Cập nhật trạng thái thiết bị, kết quả nhận diện và lịch sử cho ăn lên Firebase để ứng dụng theo dõi gần thời gian thực.

- Cảnh báo khi thức ăn sắp hết, thiết bị mất kết nối, lệnh thực hiện thất bại hoặc phát hiện đối tượng không phù hợp.

Luồng xử lý dự kiến bắt đầu khi người dùng nhấn Feed Now, khi đến thời điểm của một lịch đã lưu hoặc khi cảm biến phát hiện thú cưng trước máng ăn. Yêu cầu cho ăn được ghi vào Firebase với mã lệnh, thời gian, khẩu phần và trạng thái pending. ESP32 theo dõi lệnh mới, kiểm tra điều kiện an toàn và điều khiển servo. Trong trường hợp cần nhận diện, ESP32-S3 CAM xử lý ảnh và gửi về speciesId cùng confidence. Khi kết quả đạt ngưỡng tin cậy và phù hợp với hồ sơ đã cấu hình, bộ điều khiển mới cấp thức ăn. Load Cell đo khối lượng thực tế, sau đó thiết bị cập nhật trạng thái completed hoặc failed và ghi dữ liệu vào lịch sử.

Về tổ chức dữ liệu, thông tin cá nhân, hồ sơ loài, lịch cho ăn và thông báo được lưu theo mã uid của người dùng. Trạng thái thiết bị, kết quả nhận diện, lệnh cho ăn và lịch sử hoạt động được lưu theo mã thiết bị, chẳng hạn feeder-01. Cách phân tách này giúp ứng dụng quản lý dữ liệu riêng của từng tài khoản, đồng thời tạo một vùng dữ liệu chung để thiết bị và ứng dụng trao đổi lệnh. Khi triển khai thực tế, hệ thống cần bổ sung quan hệ sở hữu giữa uid và mã thiết bị, xác thực riêng cho thiết bị và các quy tắc Firebase giới hạn quyền đọc, ghi.

Trong phạm vi hiện tại, ứng dụng React Native và các chức năng đăng nhập, đọc, ghi dữ liệu Firebase được sử dụng làm nền tảng giao tiếp thật. Khối phần cứng và AI được thiết kế để tích hợp thông qua các trường dữ liệu đã xác định; kết quả nhận diện, trạng thái servo và số đo Load Cell cần được kiểm thử trên thiết bị thật trước khi kết luận độ chính xác toàn hệ thống. Vì vậy, quá trình đánh giá không chỉ xem xét giao diện ứng dụng mà còn phải đo sai số khẩu phần, thời gian phản hồi, độ chính xác nhận diện, độ ổn định kết nối và khả năng xử lý tình huống lỗi.

3. Mục tiêu của đề tài

Đề tài hướng đến xây dựng một hệ thống cho thú cưng ăn thông minh, trong đó khả năng nhận diện và điều khiển được thực hiện trực tiếp trên thiết bị biên.

Trong phạm vi triển khai, hệ thống tập trung vào hai loài là chó và mèo. ESP32-S3 CAM đóng vai trò bộ xử lý trung tâm, tiếp nhận hình ảnh từ camera và thực hiện suy luận mô hình AI để trả về loài dự đoán cùng độ tin cậy. Kết quả này được firmware sử dụng để lựa chọn bộ cấp thức ăn tương ứng. Quá trình cấp thức ăn được thực hiện bằng

servo, trong khi Load Cell kết hợp HX711 liên tục phản hồi khối lượng để dừng quá trình xả khi đạt giá trị mục tiêu. Đồng thời, firmware tích hợp các cơ chế như timeout, phát hiện không tăng khối lượng và thời gian chờ giữa hai lần cho ăn nhằm hạn chế lỗi vận hành và tránh cấp thức ăn lặp lại trong thời gian ngắn.

Bên cạnh chức năng điều khiển tại thiết bị, hệ thống còn kết nối với Firebase để đồng bộ cấu hình, lệnh điều khiển, trạng thái thiết bị và lịch sử cho ăn với ứng dụng Android phát triển bằng React Native. Qua đó, đề tài xây dựng được một quy trình hoàn chỉnh từ nhận diện thú cưng, ra quyết định cấp thức ăn, điều khiển phần cứng đến giám sát và quản lý dữ liệu trên ứng dụng di động, đồng thời đánh giá khả năng ứng dụng Edge AI trong một hệ thống IoT nhúng thực tế.

ID	Chức năng	Yêu cầu
FR-01	Chụp ảnh	ESP32-S3 CAM phải chụp ảnh khi luồng cho ăn được kích hoạt.
FR-02	Phân loại loài thú cưng	Mô hình Edge AI phải phân loại ảnh đã chụp vào một trong các lớp loài được hỗ trợ.
FR-03	Chọn cấu hình cho ăn	Hệ thống phải chọn cấu hình cho ăn tương ứng với loài được AI dự đoán.
FR-04	Cấp thức ăn tự động	Bộ điều khiển phải điều khiển servo để cấp lượng thức ăn đã cấu hình.
FR-05	Kiểm tra khối lượng	Hệ thống phải đo lượng thức ăn đã cấp bằng load cell/HX711 và dừng khi đạt mục tiêu trong phạm vi sai số được định nghĩa khi triển khai.
FR-06	Cho ăn thủ công	Ứng dụng di động phải có lệnh cho ăn thủ công để kiểm thử và vận hành có kiểm soát.
FR-07	Cài đặt theo loài	Ứng dụng phải cho phép cấu hình loại thức ăn, khẩu phần, lịch, giới hạn theo ngày và trạng thái auto-feed cho từng loài được hỗ trợ.
FR-08	Đồng bộ dữ liệu	Firebase phải lưu và đồng bộ cấu hình, trạng thái thiết bị và lịch sử cho ăn.
FR-09	Lịch sử & giám sát	Ứng dụng phải hiển thị các lần cho ăn gần đây, loài được nhận diện, lượng thức ăn và trạng thái thiết bị.
FR-10	Thông báo	Ứng dụng cần hỗ trợ thông báo cho các sự kiện quan trọng như hoàn tất cho ăn, sắp hết thức ăn hoặc trạng thái lỗi/thiết bị khi khả thi.

Thiết kế Edge AI

AI là chức năng cốt lõi của dự án. Mô hình chỉ chịu trách nhiệm phân loại loài thú cưng xuất hiện trước camera; quyết định cho ăn sau đó dựa trên cấu hình đã thiết lập cho loài tương ứng.

Nhiệm vụ	Phân loại ảnh đa lớp

Số lớp mục tiêu	Tối đa khoảng 5 loài thú cưng định nghĩa trước; (danh sách chính xác sẽ chốt trước khi thu thập dataset)
Vị trí suy luận	ESP32-S3 CAM (Edge AI)
Input	Ảnh camera, được resize và normalize cho mô hình
Output	Loài dự đoán + confidence score
Đề xuất mô hình	Tiny CNN hoặc mô hình MobileNet gọn nhẹ
Hướng tối ưu	INT8 quantization / TensorFlow Lite Micro hoặc stack inference tương thích ESP32

Quy tắc thiết kế quan trọng: thêm một thú cưng mới thuộc loài đã hỗ trợ không cần huấn luyện lại mô hình. Chỉ cần huấn luyện lại khi bổ sung một loài mới mà mô hình hiện tại chưa hỗ trợ.

Màn hình / Chức năng	Mục đích
Dashboard	Hiển thị trạng thái thiết bị, kết quả nhận diện mới nhất, lần cho ăn gần nhất và điều khiển nhanh.
Cài đặt theo loài	Cấu hình profile cho ăn cho từng loài được hỗ trợ.
Cho ăn thủ công	Gửi lệnh cho ăn thủ công có kiểm soát.
Lịch sử cho ăn	Hiển thị thời gian, loài, lượng mục tiêu, lượng thực tế và kết quả.
Thống kê	Tổng hợp lượng thức ăn và số lần cho ăn theo thời gian.
Thông báo	Hiển thị các cảnh báo quan trọng của hệ thống và quá trình cho ăn.
Cài đặt thiết bị	Cấu hình kết nối/thiết bị cơ bản khi cần.

4. Nội dung nghiên cứu

- Để đạt được các mục tiêu đã đề ra, nội dung nghiên cứu tập trung vào việc xây dựng và tích hợp hoàn chỉnh các thành phần của hệ thống.
- Xác định kiến trúc tổng thể và luồng dữ liệu giữa camera, ESP32-S3 CAM, mô hình Edge AI, khối điều khiển cấp thức ăn, servo, Load Cell/HX711, Firebase và ứng dụng React Native.
- Chuẩn bị dữ liệu ảnh chó và mèo, huấn luyện mô hình phân loại nhẹ, tối ưu mô hình để phù hợp với tài nguyên của ESP32-S3 CAM và tích hợp chức năng suy luận vào firmware.

- Phần cứng được thiết kế với hai bộ cấp thức ăn độc lập, trong đó gồm Servo và Load Cell dành riêng cho từng loại thú cưng. LED và buzzer được bổ sung để phản hồi trạng thái hoạt động. Firmware trên ESP32-S3 CAM được xây dựng theo dạng module, bao gồm camera, servo, HX711, hiệu chuẩn cảm biến, Feeding Controller, Wi-Fi, Firebase, quản lý trạng thái và các cơ chế xử lý lỗi. Thuật toán cấp thức ăn được thiết kế theo trình tự nhận diện loài, kiểm tra điều kiện cho ăn, lấy khẩu phần tương ứng, đo khối lượng ban đầu, mở servo và theo dõi lượng thức ăn tăng thêm cho đến khi đạt giá trị mục tiêu thì đóng servo.
- Xây dựng Firebase Realtime Database để lưu trữ và đồng bộ cấu hình, lệnh điều khiển, trạng thái thiết bị và lịch sử cho ăn. Ứng dụng Android phát triển bằng React Native được sử dụng để cấu hình hệ thống, gửi lệnh cho ăn thủ công, theo dõi trạng thái và xem lịch sử hoạt động.
- Tổ chức quá trình nhận diện thú cưng, cấp thức ăn, cập nhật dữ liệu lên Firebase đến hiển thị kết quả trên ứng dụng di động.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan hệ thống IoT và Edge AI

2.2. ESP32-S3 và ESP32-S3 CAM

2.3. Cơ sở lý thuyết về phân loại ảnh

Phân loại ảnh là bài toán trong lĩnh vực thị giác máy tính, trong đó mục tiêu là xác định ảnh đầu vào thuộc về một trong các lớp đã được định nghĩa trước. Với một ảnh đầu vào x, mô hình phân loại thực hiện ánh xạ: $f_\theta: x \rightarrow$ ŷ trong đó fθ là mô hình có tham số θ, còn ŷ là nhãn dự đoán của ảnh. Đối với bài toán phân loại C lớp, mô hình thường tạo ra một vector xác suất:

$$p = [p_1, p_2, \dots, pC]$$

với: $\sum_{i=1}^{c} p_i = 1$. Lớp dự đoán cuối cùng được xác định bằng:

$$\hat{y} = arg\ max_i\ p_i$$

Trong các hệ thống phân loại ảnh hiện đại, mạng nơ-ron tích chập (Convolutional Neural Network – CNN) được sử dụng rộng rãi nhờ khả năng tự động học các đặc trưng trực tiếp từ dữ liệu ảnh thay vì phải thiết kế đặc trưng thủ công. Ảnh đầu vào thường được biểu diễn dưới dạng tensor $X \in \mathbb{R}^{H \times W \times c}$. Trong đó H và W lần lượt là chiều cao và chiều rộng của ảnh, còn C là số kênh màu. Đối với ảnh RGB, C = 3. Trong CNN, lớp tích chập thực hiện phép toán giữa ảnh hoặc feature map đầu vào với các kernel có thể học được. Với một kernel K, giá trị tại vị trí (i, j) của feature map đầu ra biểu diễn:

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n) + b$$

trong đó b là hệ số bias. Các kernel được cập nhật trong quá trình huấn luyện để phát hiện những đặc trưng có ý nghĩa trong ảnh. Ở các tầng đầu, mạng thường học các đặc trưng đơn giản như cạnh, đường thẳng và màu sắc. Khi đi qua các tầng sâu hơn, các đặc trưng này được kết hợp để tạo thành những đặc trưng phức tạp hơn như hình dạng, cấu trúc hoặc các bộ phận đặc trưng của đối tượng. Sau lớp tích chập, hàm kích hoạt phi tuyến thường được sử dụng để tăng khả năng biểu diễn của mạng. Một trong những hàm phổ biến nhất là ReLU:

$$ReLU(x) = max(0, x)$$

Ngoài ra, các lớp pooling có thể được sử dụng nhằm giảm kích thước không gian của feature map, giảm số lượng phép tính và tăng tính bất biến đối với sự thay đổi nhỏ về vị trí của đối tượng. Max Pooling được xác định bằng cách lấy giá trị lớn nhất trong vùng:

$$Y(i,j) = max_{(m,n)} \in R\ X(i+m, j+n)$$

với R là cửa sổ pooling. Sau nhiều lớp trích xuất đặc trưng, mạng thu được tensor đặc trưng $F \in \mathbb{R}^{H' \times W' \times D}$. trong đó D là số lượng kênh đặc trưng. Tensor này cần được chuyển thành vector đặc trưng trước khi thực hiện phân loại. Một phương pháp phổ biến là Global Average Pooling (GAP). Với mỗi kênh k, GAP tính giá trị trung bình trên toàn bộ không gian:

$$z_k = 1/(H'W') \sum_{i=1}^{H'} \sum_{j=1}^{W'} F_{ijk}$$

Kết quả thu được vector $z \in \mathbb{R}^D$. Vector đặc trưng sau đó được đưa qua lớp Fully Connected để tạo ra các giá trị đầu ra, còn gọi là logits:

$$o = Wz + b$$

trong đó W và b là các tham số được học trong quá trình huấn luyện. Đối với bài toán phân loại nhiều lớp, các logits được chuyển thành xác suất bằng hàm Softmax:

$$p_i = \frac{e^{oi}}{\sum_{j=1}^{c} e^{oj}}$$

Trong quá trình huấn luyện, Cross-Entropy Loss thường được sử dụng để đánh giá sai lệch giữa nhãn thực tế và xác suất dự đoán:

$$L = -\sum_{i=1}^{c} y_i \log(p_i)$$

trong đó y_i là nhãn thực tế dưới dạng one-hot và p_i là xác suất dự đoán của lớp thứ i. Các tham số của mạng được tối ưu bằng thuật toán lan truyền ngược (Backpropagation). Gradient của hàm mất mát đối với từng tham số được tính toán và cập nhật theo quy tắc:

$$\theta_{t+1} = \theta_t - \eta \nabla \theta L$$

trong đó η là learning rate. Toàn bộ quá trình phân loại ảnh có thể được mô tả theo chuỗi:

Ảnh đầu vào → Trích xuất đặc trưng → Feature Map → Vector đặc trưng → Bộ phân loại → Xác suất → Nhãn dự đoán

Để đánh giá mô hình phân loại ảnh, các chỉ số phổ biến gồm Accuracy, Precision, Recall và F1-score. Accuracy biểu diễn tỷ lệ mẫu được dự đoán chính xác:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

Precision biểu diễn mức độ chính xác của các mẫu được mô hình dự đoán là dương:

$$Precision = TP / (TP + FP)$$

Recall phản ánh khả năng phát hiện đúng các mẫu dương thực tế:

$$Recall = TP / (TP + FN)$$

F1-score là trung bình điều hòa giữa Precision và Recall:

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Ngoài độ chính xác, đối với các hệ thống triển khai trên thiết bị nhúng hoặc Edge AI, cần xem xét thêm các yếu tố như số lượng tham số, kích thước mô hình, số phép tính MACs/FLOPs, dung lượng bộ nhớ và thời gian suy luận. Do đó, mục tiêu không chỉ là đạt độ chính xác cao mà còn phải cân bằng giữa hiệu năng phân loại và chi phí tính toán của mô hình.

2.4. Mô hình AI nhẹ cho Edge Device

Các mạng nơ-ron tích chập nhẹ được sử dụng rộng rãi cho bài toán phân loại ảnh trên thiết bị biên nhờ có số lượng tham số thấp, chi phí tính toán nhỏ và kích thước mô hình gọn. MobileNetV2 sử dụng inverted residual block và linear bottleneck, kết hợp với depthwise separable convolution nhằm giảm số lượng phép tính nhưng vẫn duy trì khả năng trích xuất đặc trưng. Dựa trên kiến trúc này, mô hình PetMobileNetV2-S3 được thiết kế với ảnh đầu vào 96x96, width multiplier 0.35, giảm số kênh đặc trưng cuối và thay lớp phân loại ImageNet bằng bốn lớp: Pet A, Pet B, Unknown và No Pet. MobileNetV3-Small tiếp tục cải tiến MobileNetV2 bằng cách kết hợp inverted residual, squeeze-and-excitation và tối ưu kiến trúc theo phần cứng. ShuffleNetV2 tập trung vào hiệu quả suy luận thực tế thông qua việc giảm chi phí truy cập bộ nhớ, kết hợp channel split và channel shuffle. Mô hình PetShuffleNetV2-S3 được xây dựng từ ShuffleNetV2-0.5×, sử dụng ảnh đầu vào 96×96 và thay lớp phân loại 1000 lớp bằng bốn lớp.

Trong những năm gần đây, nhiều kiến trúc thị giác nhẹ được phát triển nhằm cải thiện sự cân bằng giữa độ chính xác, độ trễ và chi phí tính toán trên thiết bị biên. MobileNetV4 giới thiệu Universal Inverted Bottleneck (UIB) và cơ chế Mobile MQA, hướng đến khả năng hoạt động hiệu quả trên nhiều loại phần cứng di động khác nhau. RepViT được đề xuất tại CVPR 2024 theo hướng thiết kế lại CNN nhẹ dựa trên các nguyên lý hiệu quả của Vision Transformer. Mô hình cho thấy CNN vẫn có khả năng đạt tỷ lệ accuracy–latency cạnh tranh trên thiết bị khi được tối ưu đúng kiến trúc.

Ở hướng Transformer nhẹ, SHViT sử dụng **Single-Head Attention** kết hợp thiết kế macro tối ưu bộ nhớ nhằm giảm dư thừa tính toán của Multi-Head Attention. Kiến trúc này đạt hiệu quả tốt về cả độ chính xác và latency trên CPU, GPU và thiết bị di động. Bên cạnh CNN và Transformer, Vision Mamba sử dụng Bidirectional State Space Model để học biểu diễn ảnh mà không phụ thuộc vào cơ chế self-attention truyền thống. Kiến trúc này cho thấy tiềm năng giảm chi phí bộ nhớ và tính toán, đặc biệt đối với ảnh có độ phân giải lớn. Gần đây, NuWa được giới thiệu tại CVPR 2026 nhằm tạo ra các Vision Transformer nhỏ, chuyên biệt theo từng nhóm lớp cho thiết bị biên. Phương pháp này tập trung loại bỏ các trọng số không cần thiết và tạo mô hình nhỏ hơn phù hợp với các yêu cầu tài nguyên khác nhau của Edge Device. Từ các nghiên cứu trên, MobileNetV2 có thể được sử dụng làm baseline truyền thống, trong khi MobileNetV4, RepViT, SHViT và các kiến trúc mới như Vision Mamba hoặc NuWa được sử dụng để đại diện cho các hướng phát triển hiện đại. Trong nghiên cứu này, các mô hình được so sánh dựa trên Accuracy, Macro-F1, số lượng tham số, MACs, kích thước mô hình, thời gian suy luận và mức sử dụng bộ nhớ nhằm lựa chọn kiến trúc phù hợp cho triển khai trên thiết bị Edge.

2.5. Tối ưu mô hình trên ESP32-S3

Việc triển khai mô hình học sâu trên ESP32-S3 gặp hạn chế về bộ nhớ, năng lực tính toán và mức tiêu thụ năng lượng. Do đó, các nghiên cứu về Edge AI thường tập trung vào các kỹ thuật giảm độ phức tạp của mô hình nhưng vẫn duy trì độ chính xác ở mức chấp nhận được. Một trong những kỹ thuật phổ biến nhất là **quantization**, trong đó trọng số và activation được chuyển từ FP32 sang các định dạng có độ chính xác thấp hơn như INT8. Phương pháp này giúp giảm đáng kể kích thước mô hình và lượng bộ nhớ sử dụng, đồng thời tăng tốc quá trình suy luận trên phần cứng có hỗ trợ phép toán

số nguyên. Đối với ESP32-S3, ESP-DL hỗ trợ các cấu hình lượng tử hóa như W8A8, W16A16 và mixed precision, trong đó INT8 thường được ưu tiên cho các ứng dụng yêu cầu mô hình nhỏ và thời gian suy luận thấp. Bên cạnh lượng tử hóa, việc giảm độ phân giải ảnh đầu vào cũng là một phương pháp hiệu quả để giảm chi phí tính toán. Với các mô hình CNN, số lượng phép toán phụ thuộc mạnh vào kích thước feature map, do đó việc giảm input từ 224x224 xuống 96x96 có thể làm giảm đáng kể số lượng MACs và bộ nhớ activation. Các kiến trúc sử dụng Depthwise Separable Convolution, inverted residual hoặc channel shuffle cũng được sử dụng nhằm giảm số lượng tham số và phép toán. Những kỹ thuật này là nền tảng của các mô hình nhẹ như MobileNetV2, MobileNetV3 và ShuffleNetV2.

Ngoài kích thước mô hình, mức sử dụng bộ nhớ trong quá trình suy luận cũng cần được xem xét. Activation và tensor trung gian có thể chiếm lượng RAM lớn hơn trọng số của mô hình, đặc biệt khi xử lý feature map có kích thước lớn. Vì vậy, khả năng triển khai thực tế cần được đánh giá trực tiếp dựa trên kích thước mô hình, latency, RAM/PSRAM và độ chính xác sau lượng tử hóa. Các kỹ thuật như giảm độ phân giải đầu vào, sử dụng kiến trúc CNN nhẹ và lượng tử hóa INT8 đóng vai trò quan trọng trong việc đưa mô hình phân loại ảnh xuống các thiết bị Edge có tài nguyên hạn chế.

2.6. Servo Motor và cơ cấu cấp thức ăn

2.7. Load Cell và HX711

2.8. Firebase

Firebase là nền tảng dịch vụ đám mây được sử dụng làm backend cho ứng dụng AI Smart Pet Feeder. Trong hệ thống này, Firebase đảm nhiệm ba chức năng chính: xác thực người dùng, lưu trữ và đồng bộ dữ liệu theo thời gian thực, đồng thời hỗ trợ gửi thông báo đến thiết bị di động. Việc sử dụng Firebase giúp giảm khối lượng xây dựng máy chủ riêng và tạo ra một kênh trao đổi dữ liệu chung giữa ứng dụng React Native và thiết bị ESP32. Firebase Authentication được sử dụng để thực hiện chức năng đăng ký, đăng nhập và duy trì phiên người dùng bằng email và mật khẩu. Sau khi xác thực thành công, mỗi tài khoản được cấp một mã định danh duy nhất uid. Mã này được sử dụng để phân tách hồ sơ thú cưng, lịch cho ăn và thông báo của từng người dùng, hạn chế việc một tài khoản truy cập dữ liệu riêng của tài khoản khác [1]. Firebase Realtime Database là cơ sở dữ liệu NoSQL lưu dữ liệu dưới dạng cây JSON. Khi một giá trị trong cơ sở dữ

liệu thay đổi, các ứng dụng đang kết nối có thể nhận dữ liệu mới gần như ngay lập tức mà không cần liên tục gửi yêu cầu tải lại. Đặc điểm này phù hợp với hệ thống IoT cần theo dõi trạng thái thiết bị, lượng thức ăn, kết quả nhận dạng và trạng thái lệnh cho ăn theo thời gian thực [2].

Bảng 1: Cấu trúc dữ liệu Firebase của hệ thống

Đường dẫn Firebase	Nội dung
users/{uid}/profile	Thông tin tài khoản và thiết bị được liên kết
users/{uid}/species_profiles	Khẩu phần, loại thức ăn và giới hạn của từng loài
users/{uid}/feeding_schedules	Lịch cho ăn của người dùng
users/{uid}/notifications	Thông báo của tài khoản
users/{uid}/fcm_tokens	Token nhận thông báo trên điện thoại
devices/feeder-01/status	Trạng thái online, Wi-Fi và lượng thức ăn còn lại
devices/feeder-01/latest_detection	Loài và độ tin cậy do mô hình AI trả về
devices/feeder-01/feeding_commands	Các lệnh cho ăn được gửi tới ESP32
devices/feeder-01/feeding_history	Kết quả và lịch sử các lần cho ăn

Khi người dùng nhấn Feed Now, ứng dụng tạo một lệnh mới trong feeding_commands với trạng thái pending. Thiết bị ESP32 dự kiến theo dõi đường dẫn này, điều khiển servo cấp thức ăn, đo khối lượng bằng Load Cell và cập nhật trạng thái thành processing, sau đó là completed hoặc failed. Kết quả cuối cùng được lưu trong feeding_history và đồng bộ ngược về ứng dụng. Trong phiên bản hiện tại, việc đọc và ghi Firebase đã hoạt động thật, còn phản hồi từ ESP32, servo và Load Cell đang được ứng dụng mô phỏng để phục vụ quá trình kiểm thử.

Firebase Cloud Messaging, viết tắt là FCM, được sử dụng để hỗ trợ thông báo đẩy. Khi đăng nhập, ứng dụng lấy registration token của thiết bị và lưu token vào Firebase. Trong hệ thống hoàn chỉnh, Cloud Functions hoặc một backend tin cậy sẽ sử dụng token này để gửi các cảnh báo như cho ăn thành công, thiết bị mất kết nối hoặc lượng thức ăn sắp hết. FCM gồm môi trường gửi đáng tin cậy, backend FCM, lớp vận chuyển của nền tảng và SDK nhận thông báo trên thiết bị [3].

Firebase Security Rules được sử dụng để kiểm soát quyền truy cập dữ liệu. Người dùng chỉ được đọc và ghi nhánh users/{uid} khi uid trùng với tài khoản đã xác thực. Trong bản thử nghiệm, người dùng đã đăng nhập được phép truy cập thiết bị feeder-01 dùng chung. Khi triển khai thực tế, hệ thống cần bổ sung quan hệ sở hữu giữa người dùng và thiết bị, xác thực riêng cho ESP32 và giới hạn quyền ghi đối với các trường quan trọng.

2.9. React Native

React Native là framework mã nguồn mở dùng để phát triển ứng dụng di động bằng React, JavaScript hoặc TypeScript. Khác với ứng dụng web chạy trong WebView, React Native cung cấp các thành phần như View, Text, TextInput và Pressable, sau đó ánh xạ chúng thành các thành phần giao diện của nền tảng di động. Nhờ đó, ứng dụng có thể sử dụng một cơ sở mã chung nhưng vẫn tương tác với các khả năng native của Android và iOS [4]. Ứng dụng AI Smart Pet Feeder được xây dựng bằng React Native CLI phiên bản 0.86.3 và TypeScript. TypeScript bổ sung hệ thống kiểu dữ liệu tĩnh, giúp kiểm soát các kiểu dữ liệu như hồ sơ loài, lịch cho ăn, lệnh điều khiển và lịch sử hoạt động. Điều này làm giảm lỗi khi truyền dữ liệu giữa giao diện, lớp xử lý và Firebase. React Native hiện sử dụng TypeScript làm lựa chọn mặc định cho các dự án mới [5].

Bảng 2: Tổ chức mã nguồn ứng dụng React Native

Thành phần	Vai trò
screens	Chứa Dashboard, Profile, Feed Now, Schedule, History và các màn hình khác
components	Chứa các thành phần giao diện được tái sử dụng

Thành phần	Vai trò
context	Quản lý trạng thái đăng nhập, dữ liệu thiết bị và thao tác toàn ứng dụng
repositories	Tách phần giao diện khỏi quá trình đọc và ghi Firebase
firebase	Cấu hình Realtime Database và Cloud Messaging
types	Định nghĩa các kiểu dữ liệu TypeScript
utils	Chứa các hàm hỗ trợ định dạng và xử lý dữ liệu

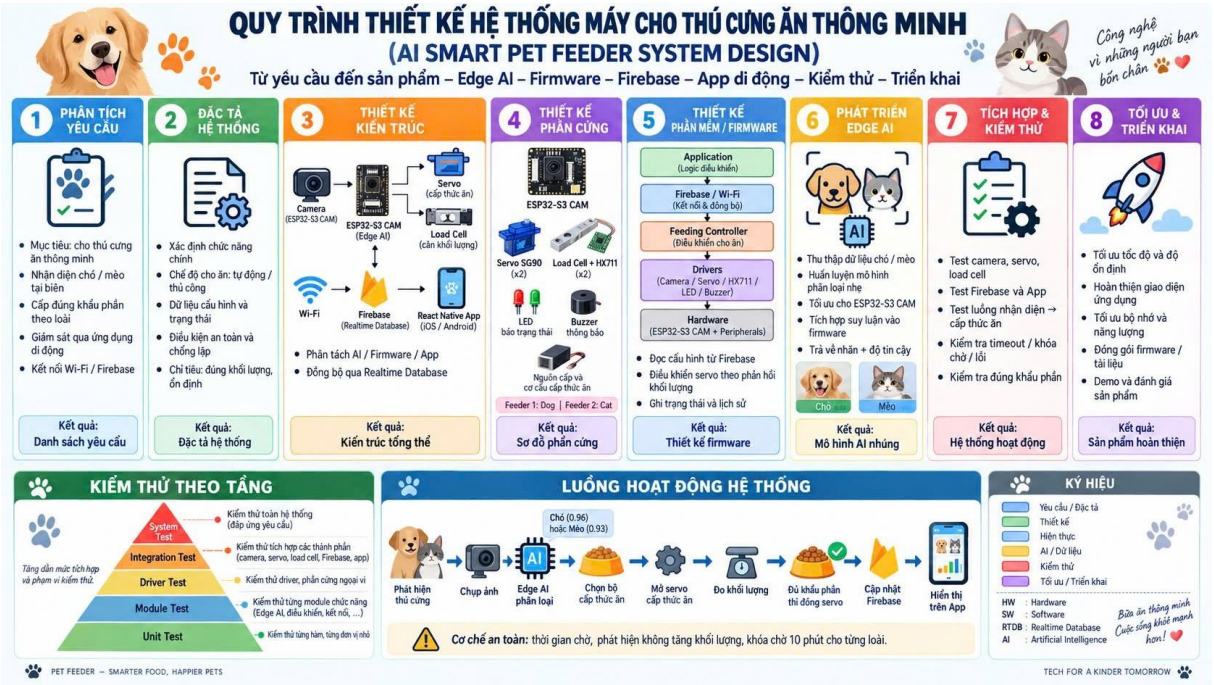
Ứng dụng sử dụng mô hình component của React. Mỗi màn hình được xây dựng từ nhiều component nhỏ và có thể tái sử dụng. Trạng thái chung được quản lý thông qua FeederContext, trong khi FirebaseFeederRepository chịu trách nhiệm đăng ký các listener và thực hiện thao tác đọc, thêm, sửa hoặc xóa dữ liệu. Cách phân lớp này giúp giao diện không phụ thuộc trực tiếp vào Firebase, thuận lợi cho kiểm thử và thay đổi backend sau này. Các module @react-native-firebase/app, auth, database và messaging tạo cầu nối giữa mã TypeScript và Firebase SDK native của Android. Project đồng thời bật Hermes và New Architecture của React Native. New Architecture sử dụng JSI, Fabric và TurboModules để cải thiện quá trình giao tiếp giữa phần JavaScript và nền tảng native, thay cho cơ chế bridge bất đồng bộ của kiến trúc cũ [6], [7].

Trong hệ thống này, React Native không trực tiếp chạy mô hình AI. Mô hình dự kiến được triển khai trên ESP32-S3 CAM để nhận dạng loài thú cưng. Ứng dụng chỉ nhận speciesId và confidence từ Firebase, hiển thị kết quả, tra cứu hồ sơ tương ứng và hỗ trợ tạo lệnh cho ăn với khẩu phần phù hợp. Việc tách AI khỏi ứng dụng di động giúp giảm tải xử lý cho điện thoại và làm rõ vai trò của từng thành phần trong hệ thống. Ưu điểm của React Native là phát triển giao diện nhanh, tái sử dụng component, hỗ trợ TypeScript và có hệ sinh thái thư viện phong phú. Tuy nhiên, các thư viện có thành phần native vẫn yêu cầu cấu hình Android SDK, Gradle, NDK và build lại ứng dụng khi thay đổi. Trong chế độ phát triển, Metro Bundler được sử dụng để đóng gói và cung cấp mã JavaScript cho ứng dụng.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Yêu cầu hệ thống

Yêu cầu hệ thống được xác định dựa trên mục tiêu xây dựng một thiết bị cho ăn thông minh có khả năng nhận diện thú cưng tại biên, điều khiển lượng thức ăn theo phản hồi khối lượng và đồng bộ dữ liệu với ứng dụng di động. Các yêu cầu được chia thành hai nhóm: yêu cầu chức năng mô tả những chức năng hệ thống phải thực hiện và yêu cầu phi chức năng quy định các đặc tính về độ ổn định, an toàn, khả năng bảo trì.



3.1.1. Yêu cầu chức năng

Các yêu cầu chức năng được xác định dựa trên chuỗi hoạt động chính của hệ thống, từ thu nhận ảnh, nhận diện thú cưng, lựa chọn bộ cấp thức ăn, điều khiển cấp theo phản hồi khối lượng đến đồng bộ dữ liệu với Firebase và ứng dụng di động. Các yêu cầu này là cơ sở để thiết kế firmware, phần cứng và các chức năng phần mềm ở các mục tiếp theo.

Bảng 3.1. Yêu cầu chức năng của hệ thống

ID	Chức năng	Yêu cầu
01	Khởi tạo và thu nhận ảnh	ESP32-S3 CAM phải khởi tạo camera thành công và cung cấp khung ảnh đầu vào cho module Edge AI.

ID	Chức năng	Yêu cầu
02	Phân loại thú cưng	Mô hình Edge AI phải phân loại ảnh thành chó, mèo hoặc không xác định và trả về độ tin cậy để firmware đánh giá.
03	Chọn bộ cấp thức ăn	Firmware phải ánh xạ chó đến bộ cấp thức ăn số 1 (Servo 1 + Load Cell 1) và mèo đến bộ cấp thức ăn số 2 (Servo 2 + Load Cell 2).
04	Đọc cấu hình khẩu phần	Hệ thống phải đọc **khối lượng thức ăn mục tiêu** từ Firebase khi có giá trị cấu hình hợp lệ; nếu không có, firmware sử dụng **khẩu phần mặc định** đã được thiết lập sẵn.
05	Cấp thức ăn theo phản hồi khối lượng	Servo phải mở để xả thức ăn và Load Cell/HX711 phải đo khối lượng tăng thêm. Servo phải đóng khi lượng thức ăn đạt gần giá trị mục tiêu.
06	Cho ăn thủ công	Ứng dụng phải có khả năng gửi lệnh cho ăn thủ công qua Firebase để ESP32 thực hiện một chu kỳ cấp thức ăn có kiểm soát.
07	Cho ăn tự động từ AI	Khi kết quả nhận diện hợp lệ và đạt **ngưỡng độ tin cậy yêu cầu**, firmware phải sử dụng **kết quả phân loại loài** để lựa chọn cơ chế cấp thức ăn tương ứng và thực hiện quá trình cho ăn tự động.
08	Thời gian chờ giữa hai lần cho ăn	Sau khi hoàn tất một lần cấp thức ăn, hệ thống phải áp dụng **khoảng thời gian chờ riêng cho từng loài** trước khi cho phép thực hiện lần cấp tiếp theo.
09	Xử lý lỗi cấp thức ăn	Firmware phải đóng servo và kết thúc chu kỳ nếu vượt timeout, Load Cell không sẵn sàng hoặc khối lượng không tăng trong khoảng thời gian cho phép.

ID	Chức năng	Yêu cầu
10	Phản hồi trạng thái tại thiết bị	LED phải sáng trong thời gian cấp thức ăn; buzzer được sử dụng để báo hoàn tất hoặc báo lỗi theo quy ước của firmware.
11	Đồng bộ Firebase	ESP32-S3 CAM phải có khả năng **đọc cấu hình và lệnh điều khiển từ Firebase**, đồng thời **cập nhật trạng thái thiết bị, trạng thái cấp thức ăn và lịch sử hoạt động** lên Firebase.
12	Giám sát trên ứng dụng	Ứng dụng React Native phải hiển thị trạng thái thiết bị, kết quả nhận diện mới nhất, cấu hình khẩu phần và lịch sử cho ăn từ Firebase.

3.1.2. Yêu cầu phi chức năng

Bên cạnh các chức năng chính, hệ thống cần đáp ứng một số yêu cầu phi chức năng để đảm bảo khả năng vận hành ổn định và thuận lợi trong quá trình phát triển, kiểm thử và mở rộng.

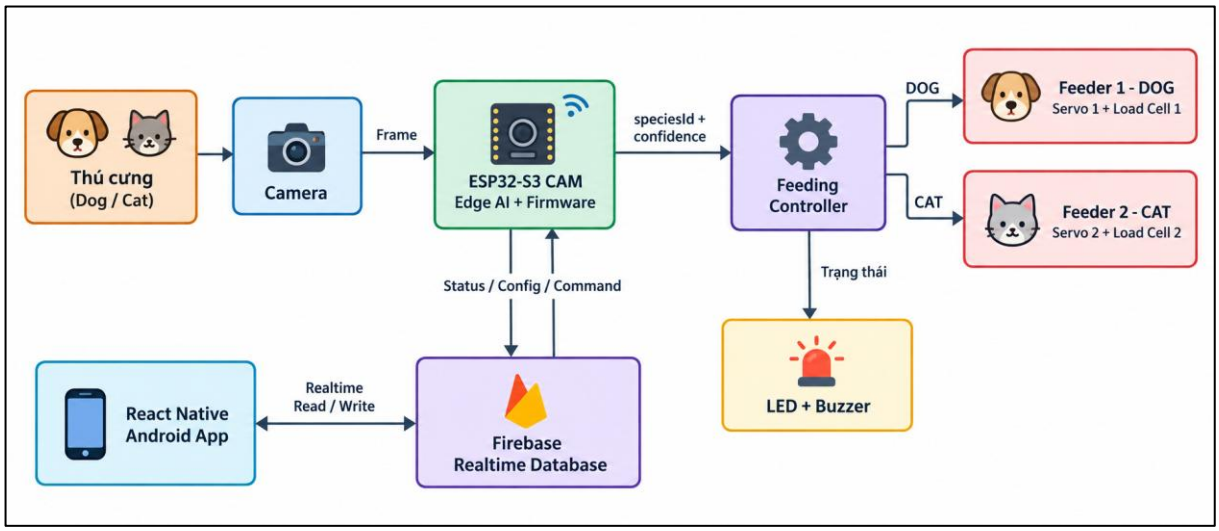
Bảng 3.2. Yêu cầu phi chức năng của hệ thống

ID	Thuộc tính	Yêu cầu
01	Độ ổn định	Các module camera, servo, HX711, Wi-Fi và Firebase phải được khởi tạo độc lập; lỗi ở một module phải được ghi nhận và không được làm servo duy trì trạng thái mở ngoài kiểm soát.
02	An toàn điều khiển	Trong mọi tình huống lỗi liên quan đến timeout, mất phản hồi khối lượng hoặc dữ liệu không hợp lệ, firmware phải ưu tiên đóng servo và tắt LED cấp thức ăn.
03	Khả năng phản hồi	Việc đọc Load Cell trong chu kỳ cấp thức ăn phải được thực hiện liên tục theo chu kỳ lấy mẫu để hệ thống có thể dừng xả khi khối lượng tiến gần mục tiêu.

ID	Thuộc tính	Yêu cầu
04	Khả năng bảo trì	Firmware phải được tổ chức theo các module độc lập như camera, AI, servo, load cell, logic cho ăn và Firebase; giao tiếp giữa AI và firmware.
05	Khả năng mở rộng	Thiết kế phải cho phép thay đổi model AI, khẩu phần hoặc ngưỡng điều khiển mà không cần viết lại toàn bộ phần điều khiển servo/load cell.
06	Khả năng hoạt động khi mạng không ổn định	Các chức năng điều khiển phần cứng cơ bản phải không phụ thuộc trực tiếp vào giao diện di động; firmware phải có cơ chế retry Wi-Fi/Firebase và dùng giá trị khẩu phần mặc định khi cấu hình từ Firebase không khả dụng.
07	Giới hạn tài nguyên Edge	Model AI và các buffer xử lý ảnh phải phù hợp bộ nhớ của ESP32-S3 CAM, tận dụng OPI PSRAM và tránh chiếm dụng tài nguyên ảnh hưởng đến điều khiển thời gian thực của cơ cấu cấp thức ăn.

3.2. Kiến trúc tổng thể hệ thống

Hệ thống được tổ chức theo kiến trúc phân lớp, trong đó ESP32-S3 CAM là nút xử lý trung tâm kết nối giữa khối nhận diện Edge AI, khối điều khiển phần cứng và nền tảng Firebase. Việc phân tách này giúp giới hạn rõ trách nhiệm của từng thành phần: AI chỉ thực hiện nhận diện; Feeding Controller chịu trách nhiệm ra quyết định cấp thức ăn; Servo và Load Cell thực thi và phản hồi quá trình cấp thức ăn; Firebase và ứng dụng di động đảm nhiệm cấu hình, đồng bộ và giám sát.



Hình 3.1. Kiến trúc tổng thể hệ thống AI Smart Pet Feeder

Theo Hình 3.1, hình ảnh từ camera được đưa vào ESP32-S3 CAM để thực hiện tiền xử lý và suy luận bằng mô hình AI **tại thiết bị biên**. Kết quả nhận diện bao gồm **loài thú cưng được dự đoán** (chó, mèo hoặc không xác định) và **độ tin cậy của kết quả nhận diện**. Mô hình AI không trực tiếp điều khiển cơ cấu cấp thức ăn mà chỉ cung cấp kết quả nhận diện cho **khối điều khiển cấp thức ăn**. Cách phân chia này giúp phần điều khiển của hệ thống vẫn có thể hoạt động ổn định ngay cả khi mô hình AI được thay đổi hoặc tối ưu trong quá trình phát triển.

Khối điều khiển cấp thức ăn đóng vai trò trung gian giữa kết quả nhận diện và các cơ cấu chấp hành. Khi hệ thống nhận diện chó, **bộ cấp thức ăn số 1**, gồm Servo 1 và Load Cell 1, được lựa chọn; khi nhận diện mèo, hệ thống sử dụng **bộ cấp thức ăn số 2**, gồm Servo 2 và Load Cell 2. Trước khi thực hiện cấp thức ăn, hệ thống kiểm tra **thời gian chờ giữa hai lần cho ăn** và xác định **khối lượng thức ăn mục tiêu**. Giá trị này được ưu tiên lấy từ cấu hình hoặc lệnh điều khiển được lưu trên Firebase; nếu không có giá trị hợp lệ, phần mềm nhúng sẽ sử dụng khẩu phần mặc định đã được thiết lập cho từng loài.

Quá trình cấp thức ăn được điều khiển dựa trên phản hồi khối lượng. Trước khi mở servo, phần mềm nhúng đo khối lượng ban đầu của khay thông qua **cảm biến Load Cell kết hợp với mạch chuyển đổi tín hiệu HX711**. Trong quá trình xả, lượng thức ăn mới được cấp được xác định bằng độ chênh lệch giữa khối lượng hiện tại và khối lượng

ban đầu. Khi lượng thức ăn tăng thêm đạt gần **khối lượng mục tiêu**, servo được đóng lại để kết thúc quá trình cấp. Cách điều khiển này giúp phần thức ăn đã có sẵn trong khay không bị tính vào lượng thức ăn mới cấp, đồng thời cho độ chính xác cao hơn so với phương pháp chỉ điều khiển servo theo một khoảng thời gian cố định.

Firebase đóng vai trò là cầu nối dữ liệu giữa ESP32-S3 CAM và ứng dụng di động phát triển bằng React Native. Dữ liệu của hệ thống được tổ chức thành các nhóm chính gồm **lệnh điều khiển, cấu hình thiết bị, trạng thái hoạt động, trạng thái thiết bị và lịch sử cho ăn**. Ứng dụng di động gửi lệnh và cập nhật cấu hình thông qua Firebase, trong khi ESP32-S3 CAM ghi lại trạng thái vận hành, kết quả cấp thức ăn và lịch sử hoạt động. Việc thống nhất cấu trúc dữ liệu giúp phần mềm nhúng và ứng dụng di động có thể được phát triển tương đối độc lập nhưng vẫn trao đổi dữ liệu theo một quy ước chung.

Ở phía thiết bị, **đèn LED** và **còi buzzer** được sử dụng để cung cấp phản hồi trực quan và âm thanh cho người dùng. Đèn LED sáng trong thời gian hệ thống đang cấp thức ăn, trong khi còi được sử dụng để báo hiệu khi quá trình cấp thức ăn hoàn tất hoặc khi xảy ra lỗi. Bên cạnh đó, phần mềm nhúng còn tích hợp các cơ chế bảo vệ như **giới hạn thời gian cấp thức ăn**, **phát hiện trường hợp khối lượng không tăng** và **thời gian chờ giữa hai lần cho ăn đối với từng loài**. Các cơ chế này giúp hạn chế tình trạng servo hoạt động quá lâu hoặc thú cưng liên tục kích hoạt quá trình cấp thức ăn trong một khoảng thời gian ngắn.

Bảng 3.3. Vai trò các khối trong kiến trúc tổng thể

Khối	**Vai trò chính**
Camera	Thu nhận hình ảnh thú cưng và cung cấp dữ liệu đầu vào cho Edge AI.
Khối Edge AI	Thực hiện tiền xử lý ảnh, suy luận mô hình và trả về **loài thú cưng được nhận diện** cùng **độ tin cậy của kết quả nhận diện**.

Khối	Vai trò chính
Khối điều khiển cấp thức ăn	Tiếp nhận kết quả nhận diện, kiểm tra điều kiện cho ăn, xác định khối lượng thức ăn mục tiêu và điều phối quá trình cấp thức ăn.
Servo + Load Cell/HX711	Servo thực hiện mở và đóng cơ cấu xả thức ăn; Load Cell kết hợp HX711 đo khối lượng để phản hồi cho hệ thống và xác định thời điểm kết thúc cấp thức ăn.
LED + Buzzer	Cung cấp tín hiệu trực quan và âm thanh để thông báo trạng thái đang cấp thức ăn, hoàn tất hoặc xảy ra lỗi.
Firebase Realtime Database	Đồng bộ lệnh điều khiển, cấu hình hệ thống, trạng thái hoạt động, trạng thái thiết bị và lịch sử cho ăn giữa ESP32-S3 CAM và ứng dụng di động.
Ứng dụng React Native	Cho phép người dùng cấu hình khẩu phần, gửi lệnh cho ăn thủ công, theo dõi trạng thái hệ thống và xem lịch sử hoạt động.

Như vậy, kiến trúc tổng thể được xây dựng theo nguyên tắc tách lớp rõ ràng giữa nhận diện, điều khiển và giám sát. Edge AI chịu trách nhiệm nhận biết loài; firmware quyết định và kiểm soát quá trình cấp thức ăn; phần cứng thực thi; Firebase và ứng dụng đảm nhiệm trao đổi dữ liệu với người dùng. Cấu trúc này phù hợp với mục tiêu triển khai Edge AI trên thiết bị nhúng và tạo nền tảng thuận lợi cho việc kiểm thử, bảo trì và mở rộng hệ thống trong các phiên bản tiếp theo.

3.3. Thiết kế phần cứng

3.3.1. ESP32-S3 CAM

3.3.2. Camera

3.3.3. Servo Motor

3.3.4. Load Cell và HX711

3.3.5. Nguồn cấp

3.3.6. Sơ đồ kết nối phần cứng

3.4. Thiết kế mô hình Edge AI

3.5. Thiết kế chức năng cho ăn theo loài

Chức năng cho ăn theo loài được thiết kế nhằm liên kết trực tiếp kết quả phân loại từ mô hình Edge AI với cơ cấu cấp thức ăn vật lý. Trong phạm vi sản phẩm hoàn thiện, hệ thống hỗ trợ hai lớp chính là chó và mèo. Mỗi lớp được ánh xạ đến một bộ cấp thức ăn độc lập, nhờ đó hệ thống có thể sử dụng loại thức ăn và khẩu phần riêng cho từng loài mà không cần nhận diện từng cá thể thú cưng.

Cấu trúc ánh xạ được thiết kế cố định ở mức phần cứng: Bộ cấp thức ăn 1 dành cho chó, sử dụng Servo 1 tại GPIO 42 và Load Cell 1 thông qua HX711 với chân DT GPIO 1, SCK GPIO 2. Bộ cấp thức ăn 2 dành cho mèo, sử dụng Servo 2 tại GPIO 41 và Load Cell 2 với DT GPIO 39, SCK GPIO 40. LED tại GPIO 48 được bật trong thời gian cấp thức ăn và buzzer tại GPIO 47 dùng để phản hồi trạng thái hoàn tất hoặc lỗi.

Bảng 3.x. Ánh xạ loài và bộ cấp thức ăn trong hệ thống

Loài	Bộ cấp	Cơ cấu xả	Phản hồi khối lượng	Khẩu phần mặc định
Chó	Bộ cấp thức ăn số 1	Servo 1 - GPIO 42	Load Cell 1 + HX711 (DT 1, SCK 2)	80 g
Mèo	Bộ cấp thức ăn số 2	Servo 2 - GPIO 41	Load Cell 2 + HX711 (DT 39, SCK 40)	45 g

Khẩu phần ưu tiên được lấy từ Firebase theo cấu hình của từng loài. Khi giá trị trên Firebase không tồn tại hoặc không hợp lệ, firmware sử dụng giá trị mặc định để bảo đảm hệ thống vẫn có thể hoạt động ở chế độ dự phòng. Trong firmware, giá trị mặc định được đặt là 80 g cho chó và 45 g cho mèo. Các giá trị này là tham số cấu hình và có thể hiệu chỉnh theo điều kiện thực tế trên ứng dụng.

Hệ thống hỗ trợ hai nguồn kích hoạt chính. Ở chế độ thủ công, ứng dụng gửi lệnh cho ăn ngay lập tức thông qua Firebase, kèm theo thông tin về loài thú cưng **và** khối lượng thức ăn mục tiêu. Ở chế độ tự động, sau khi mô hình AI nhận diện được loài với độ tin cậy đạt ngưỡng yêu cầu, firmware sử dụng kết quả nhận diện để lựa chọn bộ cấp thức ăn tương ứng. Cả hai nguồn kích hoạt đều được xử lý thông qua cùng một khối điều khiển cấp thức ăn, nhờ đó logic điều khiển servo, đo khối lượng, giới hạn thời gian xử lý và ghi lịch sử hoạt động được sử dụng thống nhất.

Để tránh việc camera nhận diện cùng một thú cưng liên tục và làm hệ thống cấp thức ăn nhiều lần, mỗi loài được thiết lập một thời gian chờ độc lập giữa hai lần cho ăn. Sau khi quá trình cấp thức ăn hoàn tất thành công, loài tương ứng sẽ tạm thời không được phép kích hoạt thêm một chu kỳ cấp thức ăn trong khoảng thời gian đã cấu hình. Cơ chế này được áp dụng riêng cho từng loài, vì vậy việc chó đang trong thời gian chờ không ảnh hưởng đến khả năng cấp thức ăn cho mèo.

Thiết kế này giúp phân tách rõ trách nhiệm giữa mô hình AI và firmware. Mô hình AI chỉ thực hiện nhiệm vụ xác định loài thú cưng và độ tin cậy của kết quả nhận diện**,** trong khi firmware chịu trách nhiệm quyết định có thực hiện cấp thức ăn hay không, lựa chọn bộ cấp tương ứng, xác định khẩu phần, kiểm tra thời gian chờ và điều khiển cơ cấu chấp hành. Cách phân tách này giúp giảm sự phụ thuộc giữa mô hình AI và phần điều khiển phần cứng, đồng thời thuận lợi khi cần thay đổi hoặc cập nhật mô hình trong tương lai.

3.6. Thiết kế thuật toán cấp thức ăn

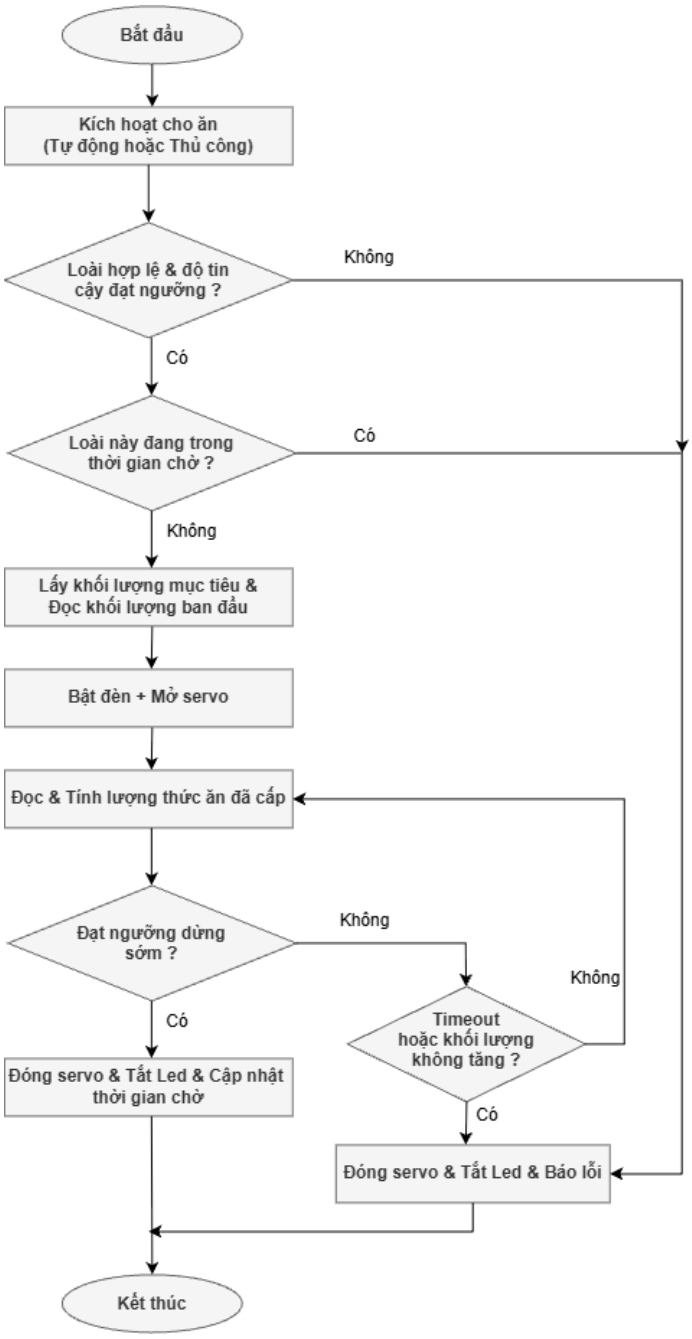
Thuật toán cấp thức ăn đóng vai trò trung tâm, tiếp nhận yêu cầu từ AI hoặc Firebase và chuyển yêu cầu đó thành một chuỗi điều khiển phần cứng có kiểm soát. Mục tiêu của thuật toán là chỉ kích hoạt đúng bộ cấp, cấp gần đúng khối lượng mục tiêu, dừng an toàn khi có lỗi và cập nhật đầy đủ trạng thái của chu kỳ cho ăn.

Đầu vào của thuật toán gồm loài thú cưng được nhận diện**,** khối lượng thức ăn mục tiêu**,** mã định danh của yêu cầu và nguồn kích hoạt quá trình cho ăn. Loài thú cưng được sử dụng để xác định bộ cấp thức ăn tương ứng, khối lượng thức ăn mục tiêu cho biết lượng thức ăn cần cấp, mã định danh giúp phân biệt từng yêu cầu để theo dõi trạng thái và lưu lịch sử, nguồn kích hoạt cho biết quá trình cho ăn được thực hiện từ thao tác thủ công trên ứng dụng hay từ cơ chế tự động dựa trên kết quả nhận diện. Nếu khối

lượng mục tiêu không được cung cấp hoặc không hợp lệ, firmware sẽ đọc khẩu phần đã cấu hình trên Firebase, nếu vẫn không có dữ liệu hợp lệ, hệ thống sử dụng giá trị mặc định được thiết lập trong firmware.

Bảng 3.x. Các đầu vào chính của thuật toán cấp thức ăn

Thông tin	Giá trị	Vai trò
Loài thú cưng được nhận diện	Chó / Mèo	Xác định bộ cấp thức ăn tương ứng
Khối lượng thức ăn mục tiêu	Số thực (g)	Xác định lượng thức ăn cần cấp; nếu không có giá trị hợp lệ thì lấy từ cấu hình
Mã định danh	Chuỗi ký tự	Phân biệt từng yêu cầu để cập nhật trạng thái và lưu lịch sử
Nguồn kích hoạt	Thủ công / Tự động	Cho biết quá trình cho ăn được kích hoạt từ ứng dụng hay từ kết quả nhận diện AI
Độ tin cậy của kết quả nhận diện	0.0 - 1.0	Được sử dụng trong chế độ tự động để kiểm tra mức độ tin cậy của kết quả AI



Hình 1: . Lưu đồ thuật toán cấp thức ăn

Thuật toán được thiết kế theo nguyên **tắc** an toàn khi xảy ra lỗi. Servo luôn được đưa về trạng thái đóng khi quá trình cấp thức ăn kết thúc, kể cả trong các trường hợp cảm biến khối lượng đọc lỗi, khối lượng không thay đổi hoặc thời gian cấp thức ăn vượt quá giới hạn cho phép. Trong phiên bản firmware hiện tại, thời gian tối đa cho một chu kỳ cấp thức ăn được thiết lập là 20 giây. Nếu hệ thống không ghi nhận khối lượng tăng tối thiểu khoảng 0,8 g trong vòng 5 giây, quá trình được xem là không có tiến triển và firmware sẽ dừng cấp thức ăn, đóng servo và ghi nhận trạng thái lỗi. Các giá trị này là

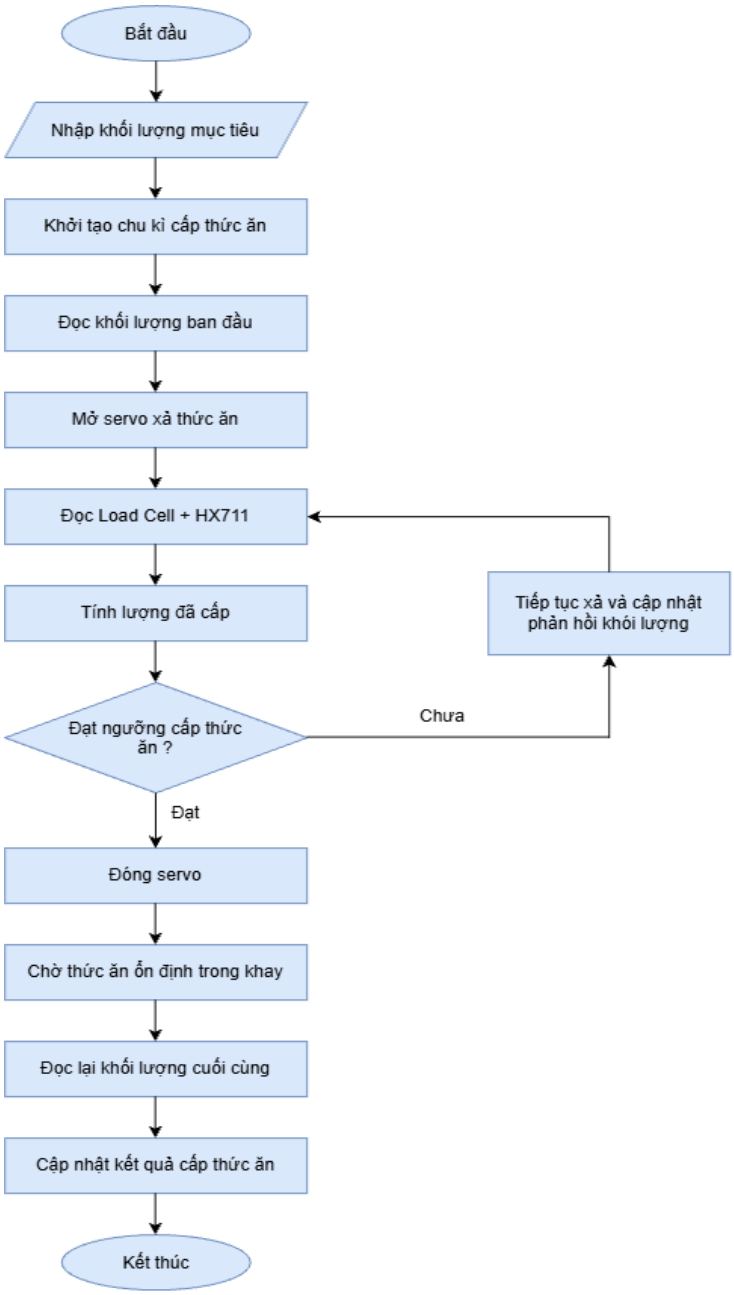
tham số triển khai ban đầu và có thể tiếp tục được hiệu chỉnh dựa trên kết quả thử nghiệm thực tế của cơ cấu cấp thức ăn.

Trong quá trình giao tiếp với Firebase, mỗi lệnh cấp thức ăn được quản lý theo các trạng thái tương ứng với chờ xử lý, đang thực hiện, hoàn thành hoặc thất bại. Đồng thời, firmware cập nhật các thông tin hoạt động của thiết bị như loài đang được xử lý, khối lượng thức ăn mục tiêu, khối lượng thực tế đã cấp và trạng thái đèn báo, cho phép ứng dụng di động theo dõi quá trình vận hành. Sau mỗi chu kỳ, hệ thống lưu một bản ghi lịch sử gồm mã định danh yêu cầu, loài thú cưng, chế độ cấp thức ăn, khối lượng mục tiêu, khối lượng thực tế, kết quả thực hiện và thông tin lỗi nếu có.

Việc sử dụng chung một thuật toán điều khiển cho cả chế độ thủ công và tự động giúp hệ thống tránh phải xây dựng hai luồng xử lý riêng biệt. Trong chế độ tự động, mô hình AI chỉ cung cấp kết quả nhận diện loài, còn các bước kiểm tra điều kiện an toàn, xác định khẩu phần và điều khiển phần cứng vẫn được firmware thực hiện thông qua khối điều khiển cấp thức ăn. Cách tổ chức này giúp firmware có cấu trúc thống nhất, thuận lợi cho quá trình kiểm thử, bảo trì và mở rộng hệ thống.

3.7. Thiết kế điều khiển dựa trên phản hồi khối lượng

Để kiểm soát lượng thức ăn đã cấp, hệ thống sử dụng cơ chế điều khiển vòng kín dựa trên phản hồi khối lượng từ Load Cell và HX711. Servo không được điều khiển theo một khoảng thời gian mở cố định, vì lưu lượng thức ăn có thể thay đổi theo kích thước hạt, mức thức ăn còn lại trong hộp, ma sát cơ khí và góc mở của cửa xả. Thay vào đó, firmware liên tục quan sát khối lượng thực tế và đóng servo khi lượng thức ăn tăng thêm đạt ngưỡng yêu cầu.



Hình 2: Lưu đồ điều khiển cấp thức ăn dựa trên phản hồi khối lượng

Trước khi mở servo, firmware đọc và lưu lại khối lượng ban đầu của khay**.** Trong quá trình cấp thức ăn, khối lượng hiện tại được cập nhật liên tục từ cảm biến Load Cell. Lượng thức ăn thực tế đã cấp được xác định bằng độ chênh lệch giữa khối lượng hiện tại và khối lượng ban đầu**:**

Lượng thức ăn đã cấp = Khối lượng hiện tại − Khối lượng ban đầu

Cách tính theo độ tăng khối lượng giúp hệ thống không bị ảnh hưởng bởi lượng thức ăn đã có sẵn trong khay trước khi bắt đầu chu kỳ. Ví dụ, nếu khay đang có 15 g và hệ thống cần cấp thêm 45 g, servo sẽ không dừng khi cân hiển thị 45 g mà chỉ dừng khi

tổng khối lượng trong khay đạt xấp xỉ 60 g. Để hạn chế lượng thức ăn tiếp tục rơi xuống do quán tính sau khi servo bắt đầu đóng, firmware sử dụng **ngưỡng dừng sớm**. Trong phiên bản hiện tại, sai số dừng sớm được đặt là 2 g. Vì vậy, servo bắt đầu đóng khi lượng thức ăn đã cấp đạt gần giá trị mục tiêu theo điều kiện:

Lượng thức ăn đã cấp ≥ Khối lượng mục tiêu − Sai số dừng sớm

Sau khi servo đóng, firmware chờ một khoảng thời gian ngắn để phần thức ăn còn lại trong máng rơi hết xuống khay, sau đó đọc lại cảm biến Load Cell để xác định khối lượng thực tế cuối cùng đã cấp. Giá trị này được dùng để cập nhật trạng thái của lần cho ăn và lưu vào lịch sử trên Firebase. Nhờ đó, hệ thống phân biệt rõ giữa ngưỡng điều khiển dùng để quyết định thời điểm đóng servo **và** khối lượng thực tế cuối cùng dùng để ghi nhận kết quả**.**

Tham số	Giá trị	Ý nghĩa
Chu kỳ đọc khối lượng	120 ms	Khoảng thời gian giữa hai lần đọc giá trị khối lượng trong quá trình cấp thức ăn
Sai số dừng sớm	2 g	Bù cho lượng thức ăn còn tiếp tục rơi xuống sau khi servo bắt đầu đóng
Mức tăng khối lượng tối thiểu	0,8 g	Ngưỡng dùng để xác định quá trình cấp thức ăn vẫn đang có tiến triển
Thời gian tối đa không ghi nhận tiến triển	5 s	Dừng quá trình cấp nếu khối lượng không tăng đủ trong khoảng thời gian quy định
Thời gian cấp thức ăn tối đa	20 s	Giới hạn thời gian tối đa cho một chu kỳ cấp thức ăn
Thời gian ổn định sau khi mở servo	500 ms	Khoảng thời gian chờ để cơ cấu ổn định sau khi servo được mở
Thời gian chờ thức ăn rơi hoàn tất	800 ms	Khoảng thời gian chờ sau khi servo đóng để phần thức ăn còn lại trong máng rơi xuống khay

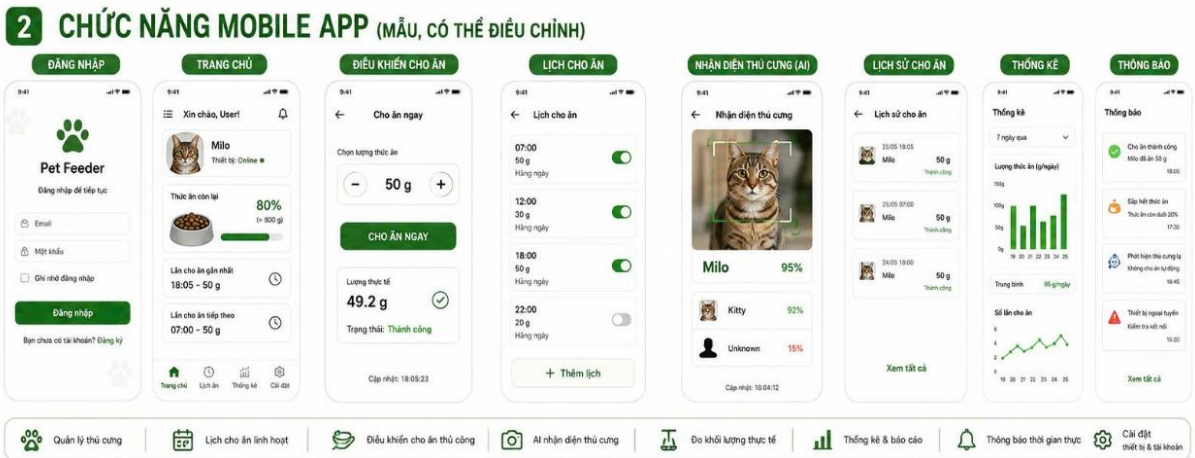
Bảng 3: Các tham số điều khiển khối lượng trong firmware

Hai cảm biến Load Cell cần được hiệu chuẩn trước khi hệ thống cho phép thực hiện quá trình cấp thức ăn tự động. Firmware hỗ trợ chức năng hiệu chuẩn thông qua cổng Serial. Trước tiên, cảm biến được đưa về trạng thái không tải để xác lập giá trị gốc, sau đó đặt một vật có khối lượng đã biết lên Load Cell để xác định hệ số chuyển đổi từ tín hiệu đo sang đơn vị khối lượng. Hệ số hiệu chuẩn được lưu trong bộ nhớ không mất dữ liệu của ESP32-S3, nhờ đó vẫn được giữ lại sau khi thiết bị khởi động lại. Nếu một

Load Cell chưa được hiệu chuẩn, firmware sẽ từ chối thực hiện chu kỳ cấp thức ăn tương ứng thay vì sử dụng trực tiếp giá trị thô từ mạch HX711. Bên cạnh điều kiện đạt khối lượng mục tiêu, quá trình điều khiển còn liên tục kiểm tra tính hợp lệ của tín hiệu phản hồi. Nếu mạch HX711 không cung cấp được dữ liệu, khối lượng không tăng đủ trong khoảng thời gian cho phép hoặc thời gian cấp thức ăn vượt quá giới hạn tối đa, firmware sẽ đóng servo ngay và kết thúc chu kỳ với trạng thái lỗi. Khi đó, buzzer phát tín hiệu cảnh báo và thông tin lỗi được cập nhật lên Firebase. Các cơ chế này giúp hạn chế nguy cơ servo duy trì trạng thái mở trong các trường hợp như hết thức ăn trong ngăn chứa, thức ăn bị kẹt hoặc cảm biến khối lượng gặp sự cố. Nhờ sử dụng phản hồi trực tiếp từ khối lượng đo được, hệ thống không phụ thuộc hoàn toàn vào thời gian quay của servo mà điều khiển dựa trên lượng thức ăn thực tế đã cấp. Cách tiếp cận này giúp cải thiện tính ổn định và khả năng lặp lại của quá trình cấp thức ăn, đồng thời cung cấp khối lượng thực tế đã cấp để ứng dụng di động hiển thị và giám sát. Các giá trị như sai số dừng sớm và giới hạn thời gian hiện tại được thiết lập theo cấu hình của firmware phiên bản đầu tiên; việc hiệu chỉnh tối ưu các tham số này cần dựa trên kết quả thử nghiệm thực tế của sản phẩm hoàn thiện.

3.8. Thiết kế Firebase

3.9. Thiết kế ứng dụng React Native



CHƯƠNG 4: PHÂN TÍCH KẾT QUẢ

4.1. Môi trường thực nghiệm

4.2. Kết quả mô hình AI

4.3. Kết quả triển khai Edge AI

4.4. Kết quả Firebase và ứng dụng

KẾT LUẬN

TÀI LIỆU THAM KHẢO